Experiment No.7
Implementation of Intermediate Code Generation (ICG) phase of compiler
Name-Vishal Pareek TE-2 44
Date of Performance:18/02/26
Date of Submission:25/02/26

Aim: Implementation of Intermediate Code Generation (ICG) phase of compiler

Objective: To develop an ability to apply and develop Intermediate Code representation techniques in Compiler construction.

Theory:

In the analysis-synthesis model of a compiler, the front end of a compiler translates a source program into an independent intermediate code, and then the back end of the compiler uses this intermediate code to generate the target code.

The benefits of using machine independent intermediate code are:

- Because of the machine independent intermediate code, portability will be enhanced. For ex, suppose, if a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required. Because, obviously, there were some modifications in the compiler itself according to the machine specifications.
- Retargeting is facilitated.
- It is easier to apply source code modification to improve the performance of source code by optimizing the intermediate code.\
- If a compiler translates the source language to its target machine language without having the option for generating intermediate code, then for each new machine, a full native compiler is required.
- Intermediate code eliminates the need of a new full compiler for every unique machine by keeping the analysis portions same for all the compilers.
- The second part of compiler, synthesis, is changed according to the target machine.

The following are commonly used intermediate code representation:

1. **Postfix Notation –**

The ordinary (infix) way of writing the sum of a and b is with operator in the middle: a + b The postfix notation for the same expression places the operator at the right end as ab +. In general, if e1 and e2 are any postfix expressions, and + is any binary operator, the result of

applying + to the values denoted by e1 and e2 is postfix notation by e1e2 +. No parentheses are needed in postfix notation because the position and arity (number of arguments) of the operators permit only one way to decode a postfix expression. In postfix notation the operator follows the operand.

Example – The postfix representation of the expression (a – b) * (c + d) + (a – b) is : ab – cd + *ab -+.

2. Three-AddressCode

A statement involving no more than three references (two for operands and one for result) is known as three address statement. A sequence of three address statements is known as three address code. Three address statement is of the form x = y op z , here x, y, z will have address (memory location). Sometimes a statement might contain less than three references, but it is still called three address statement.

Example – The three-address code for the expression a + b * c + d:

T 1 = b * c
T 2 = a + T 1
T 3 = T 2 + d

T 1, T 2, T 3 are temporary variables.

3. Quadruples

Each instruction in quadruples presentation is divided into four fields: operator, arg1, arg2, and result. The above example is represented below in quadruples format:

Op	arg_1	arg_2	result
*	c	d	r1
+	b	r1	r2
+	r2	r1	r3
=	r3		a

4. Triples

Each instruction in triples presentation has three fields: op, arg1, and arg2.The results of respective sub-expressions are denoted by the position of expression. Triples represent

similarity with DAG and syntax tree. They are equivalent to DAG while representing expressions.

Op	arg_1	arg_2
*	c	d
+	b	(0)
+	(1)	(0)
=	(2)	

Triples face the problem of code immovability while optimization, as the results are positional and changing the order or position of an expression may cause problems.

5. Indirect Triples

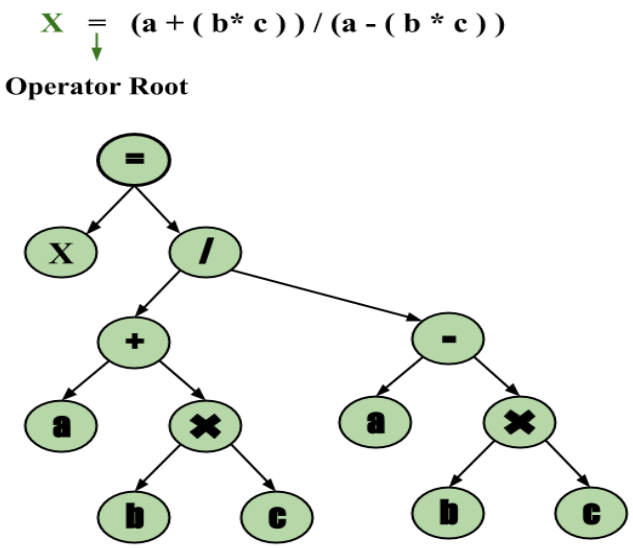
This representation is an enhancement over triples representation. It uses pointers instead of position to store results. This enables the optimizers to freely re-position the sub-expression to produce an optimized code.

6. Syntax Tree

Syntax tree is nothing more than condensed form of a parse tree. The operator and keyword nodes of the parse tree are moved to their parents and a chain of single productions is replaced by single link in syntax tree the internal nodes are operators and child nodes are operands. To form syntax tree put parentheses in the expression, this way it's easy to recognize which operand should come first.

This representation is an enhancement over triples representation. It uses pointers instead of position to store results. This enables the optimizers to freely re-position the sub-expression to produce an optimized code.

Example –
x = (a + b * c) / (a – b * c)



Note: students need to implement at least two ICG for given input statement.

Code:

```
import java.util.*;


class IntermediateCodeGeneration {


    // Function to return precedence of operators
    static int precedence(char op) {
        if (op == '+' || op == '-')
            return 1;
        if (op == '*' || op == '/')
            return 2;
        return 0;
    }
```

```
// Function to convert infix expression to postfix
static String infixToPostfix(String expression) {
    StringBuilder output = new StringBuilder();
    Stack<Character> stack = new Stack<>();

    for (char ch : expression.toCharArray()) {
        if (Character.isLetterOrDigit(ch)) {
            output.append(ch);
        } else if (ch == '(') {
            stack.push(ch);
        } else if (ch == ')') {
            while (!stack.isEmpty() && stack.peek() != '(') {
                output.append(stack.pop());
            }
            stack.pop();
        } else { // Operator
            while (!stack.isEmpty() && precedence(stack.peek()) >= precedence(ch)) {
                output.append(stack.pop());
            }
            stack.push(ch);
        }
    }

    while (!stack.isEmpty()) {
        output.append(stack.pop());
    }
```

```
    return output.toString();
}

// Function to generate Three Address Code
static List<String> generateTAC(String postfix) {
    Stack<String> stack = new Stack<>();
    List<String> tac = new ArrayList<>();
    int tempCount = 1;

    for (char ch : postfix.toCharArray()) {
        if (Character.isLetterOrDigit(ch)) {
            stack.push(String.valueOf(ch));
        } else { // Operator
            String op2 = stack.pop();
            String op1 = stack.pop();
            String tempVar = "T" + tempCount++;
            tac.add(tempVar + " = " + op1 + " " + ch + " " + op2);
            stack.push(tempVar);
        }
    }

    return tac;
}

public static void main(String[] args) {
    Scanner scanner = new Scanner(System.in);
```

```
        System.out.print("Enter an arithmetic expression: ");
        String expression = scanner.nextLine().replace(" ", "");
        scanner.close();

        // Convert to postfix
        String postfix = infixToPostfix(expression);
        System.out.println("Postfix Notation: " + postfix);

        // Generate Three Address Code
        List<String> tac = generateTAC(postfix);
        System.out.println("\nThree Address Code:");
        for (String line : tac) {
            System.out.println(line);
        }
    }
}
```

Output:

```
C:\Users\student\Downloads>javac IntermediateCodeGeneration.java

C:\Users\student\Downloads>java IntermediateCodeGeneration.java
Enter an arithmetic expression:  (a + b * c) / (a - b * c)
Postfix Notation: abc*+abc*-/

Three Address Code:
T1 = b * c
T2 = a + T1
T3 = b * c
T4 = a - T3
T5 = T2 / T4

C:\Users\student\Downloads>
```

Conclusion:

The implementation of Intermediate Code Generation (ICG) successfully converts an infix expression into its corresponding postfix notation and generates Three Address Code (TAC). This approach enhances portability and optimizes the compilation process by maintaining a machine-independent intermediate representation.

References

[1] Compilers: Principles, Techniques, and Tools, 2nd ed. Boston, MA, USA: Pearson, 2006.

[2] Engineering a Compiler, 2nd ed. Burlington, MA, USA: Morgan Kaufmann, 2011.

[3] Modern Compiler Implementation in C. Cambridge, U.K.: Cambridge University Press, 1998.

[4] GeeksforGeeks, "Intermediate Code Generation in Compiler Design." [Online]. Available: https://www.geeksforgeeks.org

[5] TutorialsPoint, "Compiler Design – Intermediate Code Generation." [Online]. Available: https://www.tutorialspoint.com